

# BookIQ: Data-Driven Book Metadata Intelligence Platform

## Technical Report

**Module:** COMP3011 — Web Services and Web Data

**Student:** Jason Tsoi

**Date:** March 20, 2026

---

### Quick Links:

**GitHub Repository:** <https://github.com/LifeIshard0/COMP3011-Web-Services-and-Web-Data-CW1>

**API Documentation:** <https://github.com/LifeIshard0/COMP3011-Web-Services-and-Web-Data-CW1>

**Live API / Swagger:** <https://web-production-bca62.up.railway.app/api/docs/>

**Presentation Slides:** <https://canva.link/801336kr41cz26c>

---

## 1 Project Overview

BookIQ is a RESTful API for managing and analysing book metadata. It supports CRUD operations, role-based access control, PostgreSQL full-text search, hybrid ranking, personalised recommendations, analytics endpoints, and an MCP server layer for AI agent integration.

The motivation for the project was that many public book APIs return raw metadata with little validation or intelligence. BookIQ improves this by cleaning records automatically, validating ISBNs, detecting likely duplicates, scoring metadata quality, and exposing better search and recommendation features.

## 2 Technical Choices

I chose **Python**, **Django REST Framework**, and **PostgreSQL** because they provided the best balance of development speed, database capability, and maintainability.

- **Django REST Framework** gave built-in serializers, routing, authentication support, and permission handling.
- **PostgreSQL** was chosen over SQLite because it supports native full-text search, GIN indexing, JSON fields, and stronger production-ready relational features.
- **JWT authentication** was used to support stateless API access.
- **Railway** was used for deployment because it provides simple GitHub-based CI/CD and managed PostgreSQL.

The system follows a layered structure: views handle HTTP requests, service modules contain business logic, and models handle persistence. This made the project easier to test and maintain.

## 3 Main Features

### 3.1 CRUD and RBAC

BookIQ supports full CRUD for books, with role-based access:

- **Reader:** view books, rate books, get recommendations
- **Curator:** reader permissions plus create/update books
- **Admin:** curator permissions plus delete books, run imports, and access analytics

### 3.2 Metadata Cleaning Pipeline

Every book write triggers a 7-step automated cleaning pipeline:

1. Title normalisation
2. Author standardisation
3. ISBN-13 validation
4. Genre inference
5. Duplicate detection
6. Quality scoring
7. Flagging low-quality records

This improves consistency and creates a curator review workflow.

### 3.3 Search

BookIQ implements PostgreSQL full-text search using weighted `SearchVector`, `SearchRank`, and a GIN index. It also includes hybrid Reciprocal Rank Fusion (RRF), combining text relevance with Wilson Score popularity.

### 3.4 Recommendations

The recommendation system uses a three-strategy waterfall:

- Content-based filtering
- Collaborative filtering
- Popularity fallback for cold-start users

Each recommendation includes the strategy used, which makes the output explainable.

### 3.5 Analytics

Admin endpoints provide:

- Genre trend analytics
- Catalogue health summary
- Import and quality statistics

### 3.6 MCP Server

The API is also exposed through an MCP-compatible server so AI agents can use BookIQ as a tool source.

## 4 Testing and Deployment

The project includes **168 automated tests** across 15 test modules, covering authentication, CRUD, search, imports, cleaning logic, recommendations, analytics, permissions, and error handling.

Testing tools used include `pytest`, `pytest-django`, and `pytest-cov`. The application is deployed on **Railway** with PostgreSQL and linked to GitHub for automatic redeployment on push.

## 5 Challenges and Limitations

A key challenge was balancing advanced functionality with coursework scope. I initially considered embedding-based semantic search and zero-shot genre classification, but removed them in favour of more transparent and assessable algorithmic approaches.

Current limitations:

- CSV import is synchronous, so very large imports would need background jobs in production.
- Genre inference uses a manually curated keyword dictionary.
- Collaborative filtering is simplified for coursework scale.

If extended further, I would add async task processing, caching, and broader recommendation signals.

## 6 Reflection

This project helped me strengthen my understanding of API design, database indexing, permission systems, testing, and deployment. The most valuable design decision was keeping business logic in service modules rather than views, which made the code cleaner and easier to test.

BookIQ goes beyond the minimum CRUD requirement by adding intelligent metadata processing, hybrid search, recommendations, analytics, and MCP integration.

## A GenAI Usage Declaration

Per COMP3011 assessment policy, all AI tool usage is declared below:

| Tool           | Purpose                                                                         | Nature                          |
|----------------|---------------------------------------------------------------------------------|---------------------------------|
| Perplexity AI  | Architecture planning, design decisions, algorithmic trade-offs, report writing | High-level creative exploration |
| GitHub Copilot | Inline code completion, bug fixing                                              | Productivity aid                |

### Design decisions analysis with GenAI:

- Eliminated semantic search (embeddings) as outside brief scope.
- Eliminated BART zero-shot genre classification in favour of transparent keyword matching.
- Chose PostgreSQL FTS over Elasticsearch to demonstrate database indexing skills.
- Designed service layer pattern before implementation.
- Built 25-commit roadmap upfront to ensure iterative delivery.

### Honest limitations identified:

- Synchronous import limits scalability (production fix: Celery + Redis).
- Genre keyword dictionary is manually curated (could be ML classifier).
- Collaborative filtering checks first 500 users only (acceptable for coursework scale).

All design choices are the student's own understanding and can be explained in an oral examination.

## Conversation Logs

### Main Questions:

*77731819-f4fa-46a6-b6e3-2f7891c5dbc0.pdf is my idea.  
COMP3011\_Coursework1\_Brief\_2025\_2026.pdf is the web assessment brief.  
I want you review my idea and generate a technical report in notion and then guide me how to finish the project step-by-step by breaking it into small parts.*

### Output (Roadmap Strategy):

#### BookIQ — Full Build Roadmap (March → May 2026)

##### Phase 1 — Foundation (Week 1-2, now)

- 01 Django project + app structure + requirements.txt
- 02 Custom User model with role field (Reader/Curator/Admin)
- 03 JWT authentication endpoints (register, login, refresh)
- 04 Book model + migrations

- 05 Book CRUD endpoints (full REST)
- 06 RBAC permission classes wired to endpoints

### **Phase 2 — Data & Intelligence (Week 3-4)**

- 07 BookRating model + rate endpoint (upsert + signals)
- 08 Metadata cleaning pipeline (7 steps)
- 09 CSV dataset import + ImportJob model
- 10 Keyword search endpoint
- 11 Semantic embeddings (sentence-transformers)
- 12 Hybrid search with RRF

### **Phase 3 — Advanced Features (Week 5-6)**

- 13 Content-based recommender
- 14 Collaborative filtering + cold start fallback
- 15 Genre trend analytics (TruncMonth + Wilson Score)
- 16 Catalogue summary analytics endpoint

### **Phase 4 — Professional Polish (Week 7-8)**

- 17 MCP server layer (bookiq\_mcp\_server.py)
- 18 Swagger UI / drf-spectacular API docs
- 19 Global error handling + consistent JSON envelope
- 20 pytest suite (4 test modules)

### **Phase 5 — Deployment & Submission (Week 9-10)**

- 21 Production settings (DEBUG=False, env vars, whitenoise)
- 22 Railway deployment + PostgreSQL wired up
- 23 README.md (professional, full setup instructions)
- 24 Final polish — pagination, ordering, filtering
- 25 API docs PDF export + Notion report evidence filled in

### **What I did:**

*Then using the main framework of the roadmap and ask copilot to make it step by step. Sometimes I have to review the idea and fix some bugs.*